

DOCUMENTACIÓN TÉCNICA PROFESIONAL — TOO-EASY

Revisión de Arquitecto de Software Senior · Tech Lead · Auditor Técnico
Versión 1.0 · 14 de junio de 2026

Resumen Ejecutivo del Ecosistema

Too-Easy es una aplicación web educativa de finanzas personales con mecánicas de gamificación. Su objetivo es que usuarios —con foco en jóvenes de 15 a 35 años— aprendan conceptos financieros básicos a través de flashcards, quizzes interactivos y un sistema de retos ligados a movimientos financieros reales. El proyecto está desplegado en Render (<https://tooeasy-8zct.onrender.com>) con base de datos en Supabase (PostgreSQL administrado) y un cliente móvil Android (APK en GitHub Releases). El stack tecnológico es: **Node.js + Express 5 + Supabase (PostgreSQL) + Vanilla JS/HTML/CSS** en el frontend. El proyecto fue desarrollado por un equipo académico de cinco personas del grupo 5IV8.

Equipo	Rol
Granados Luna Mario	Desarrollador
Adán	Desarrollador
Faro Montes Leonardo	Desarrollador
Ramírez Díaz Luz Karen	Desarrolladora
Vega Sandoval Julio Emiliano	Desarrollador

MÓDULO 1 — SERVIDOR PRINCIPAL

Archivo: `server.js`

1. Descripción General del Módulo

Constituye el punto de entrada y núcleo orquestador de toda la aplicación. Inicializa el runtime HTTP, aplica la capa de seguridad perimetral, registra las 15 rutas del API y arranca el proceso de limpieza automática de tickets.

¿Por qué existe? En una arquitectura monolítica modular, existe un único proceso Node que sirve simultáneamente el API REST y los archivos estáticos del frontend. El `server.js` es la raíz de ese árbol de dependencias.

Objetivo dentro del ecosistema: Es la capa de composición que une el plano de infraestructura (Helmet, Rate Limit, CORS) con el plano de dominio (rutas de negocio) y el plano de datos (Supabase).

2. Lógica de Negocio

Flujo completo de una petición HTTP

```
Cliente (Browser/APK)
├─ [1] TCP → Express listener (PORT 3000 / Render 443)
├─ [2] Helmet CSP + HSTS + noSniff + Frameguard
├─ [3] CORS whitelist check
├─ [4] express.json({limit:'10kb'}) → parse body
├─ [5a] /public/** → static files (SIN rate limit)
├─ [5b] /api/auth/** → authLimiter (20 req/15min) → router
├─ [5c] /api/** → globalLimiter (300 req/15min) → router
└─ [6] 404 handler / Error handler
```

Reglas de negocio clave:

- `SESSION_SECRET` ausente → proceso termina (exit 1). No hay arranque inseguro silencioso.
- `trust proxy 1` activo para que `req.ip` reciba la IP real del cliente detrás de Render/Nginx.
- Los archivos estáticos se sirven antes del rate limit para no consumir cuota de peticiones API con assets.
- El job de limpieza de tickets arranca 10 segundos después del boot y luego cada 24 horas.

3. Arquitectura Técnica

Patrón arquitectónico: Monolito modular con separación en capas (Routes → Controllers → Models → Services/Utils).

Componente	Rol
Express 5	Framework HTTP
Helmet	Cabeceras de seguridad HTTP
express-rate-limit	Throttling por IP
cors	Whitelist de orígenes permitidos
setInterval nativo	Cron job de limpieza de tickets

Justificación: Elegir un monolito modular para un equipo académico de 5 personas es correcto — reduce la complejidad operacional de microservicios sin sacrificar la organización del código.

4. Tecnologías Utilizadas

Express 5 (^5.2.1)

Framework web minimalista para Node.js. Seleccionado por ser el estándar de facto para APIs REST. Express 5 incorpora manejo nativo de errores asíncronos (async/await sin try-catch en middleware). *Alternativas evaluadas:* Fastify (30% más rápido en benchmarks, pero con mayor curva de aprendizaje), Hono (edge-native, ultraligero).

Helmet (^8.1.0)

Middleware que aplica automáticamente 14+ cabeceras HTTP de seguridad. Configuración implementada: CSP con whitelist explícita (self + Cloudflare Turnstile), HSTS con maxAge 1 año + preload, X-Frame-Options: DENY, X-Content-Type-Options: noSniff, Referrer-Policy: strict-origin-when-cross-origin.

express-rate-limit (^8.4.1)

Middleware de throttling. Dos limitadores: global (300 req/15min) y auth (20 req/15min). *Limitación crítica:* Sin backend persistente como Redis, los contadores se pierden al reiniciar el servidor. En Render free tier con spin-down frecuente, el rate limiting pierde efectividad real.

5. Algoritmos y Procesos Internos

Cron Job de Limpieza de Tickets

```
setTimeout(runTicketCleanup, 10_000); // primera ejecución: t+10s
setInterval(runTicketCleanup, 24*60*60*1000); // repetición: cada 24h
```

El algoritmo consulta tickets con `created_at < now()-30days`, borra las imágenes del bucket de Supabase Storage y elimina los registros. La operación es idempotente: si falla a mitad, los tickets no borrados se reintentan en la siguiente ejecución.

Problema identificado: El `setTimeout` inicial de 10 segundos es un workaround frágil. Si Supabase tarda más en aceptar conexiones (cold start), el primer cleanup fallará silenciosamente.

6. Seguridad

Control	Implementación	Calificación
CSP	Directivas explícitas, unsafe-inline ausente	Buena
HSTS	1 año + preload	Excelente
Clickjacking	frameguard: deny	Excelente
MIME Sniffing	noSniff: true	Excelente
CORS	Whitelist explícita con credentials: true	Buena
Rate Limiting	Dos niveles pero sin persistencia Redis	Regular
Body Size	10kb límite estricto	Buena
SESSION_SECRET	Verificado al boot, proceso aborta si falta	Excelente

⚠ Vulnerabilidad Crítica — Variables de entorno expuestas: El archivo `.env` contiene en texto plano el `SUPABASE_SERVICE_ROLE_KEY` (privilegios de superadmin que bypassan RLS), `RESEND_API_KEY`, `BREVO_API_KEY`, `EMAIL_PASS`, `TURNSTILE_SECRET_KEY`, `SESSION_SECRET` y `ALPHA_VANTAGE_API_KEY`. Todas estas claves **deben ser rotadas inmediatamente** si el repositorio es o fue público.

7. Rendimiento

- Los archivos estáticos se sirven antes del rate limiter: decisión correcta que evita overhead innecesario.
- **Sin compresión gzip** habilitada (middleware `compression` ausente). En producción esto representa una transferencia 60-80% mayor para HTML/CSS/JS.
- Sin cabeceras `Cache-Control` en archivos estáticos.

8. Escalabilidad

Cuello de botella principal: El cron job de tickets en memoria. Con múltiples instancias, cada una ejecutaría el cleanup independientemente generando race conditions sobre los mismos tickets.

Solución recomendada: Usar `pg_cron` en Supabase o BullMQ con Redis para scheduling distribuido.

Limitación de Render free tier: El servidor hace spin-down después de 15 minutos de inactividad. El `setInterval` de 24h no sobrevivirá — en la primera petición tras el spin-down el servidor arranca de nuevo y el intervalo se reinicia.

9. Mejoras Futuras

Horizonte	Mejora	Impacto
Corto plazo	Agregar middleware <code>compression()</code>	-60-80% en transferencia
Corto plazo	Cache-Control en <code>express.static</code>	Mejora de TTFB en visitas repetidas
Corto plazo	Rotar todas las claves del <code>.env</code>	Seguridad crítica
Mediano plazo	Migrar cron job a <code>pg_cron</code> en Supabase	Sobrevive reinicios del servidor
Mediano plazo	Redis (Upstash free tier) para rate limiting	Rate limiting efectivo
Largo plazo	Separar API de archivos estáticos (CDN)	Escalabilidad horizontal

10. Evaluación Profesional — Módulo 1

Criterio	Calificación	Justificación
Arquitectura	7/10	Monolito modular bien estructurado; falta compresión y cache headers
Seguridad	7/10	Helmet y CORS sólidos; rate limiter sin Redis pierde efectividad real
Escalabilidad	5/10	Sin Redis, cron en memoria, sin separación API/static
Mantenibilidad	8/10	Código limpio, comentado, estructura clara
Rendimiento	5/10	Sin compresión, sin cache estático

MÓDULO 2 — SISTEMA DE AUTENTICACIÓN

Archivos: `authController.js` · `security.js` · `requireAuth.js` · `authRecoveryController.js`

1. Descripción General del Módulo

Gestiona el ciclo de vida completo de la identidad del usuario: registro con verificación de email, inicio de sesión con protección ante fuerza bruta, recuperación de contraseña en 3 pasos, y validación de sesión en cada petición autenticada.

Objetivo: Producir un `sessionToken` HMAC firmado que todos los demás módulos verifican para autorizar peticiones. Garantiza que solo usuarios legítimos —con email verificado y credenciales válidas— puedan acceder a la plataforma.

2. Lógica de Negocio

Flujo de Registro

```
[1] Turnstile anti-bot → [2] Validaciones de campo → [3] Unicidad email/nombre
    → [4] HIBP pwned check → [5] bcrypt hash (12 rounds)
    → [6] INSERT usuario (email_verified=false)
    → [7] Envío código 6 dígitos por email
    → [8] Respuesta: requiresVerification=true
```

Flujo de Login

```
[1] Turnstile → [2] Lookup usuario por email
    → [3] Bcrypt dummy si no existe (timing attack prevention)
    → [4] Verificar bloqueo temporal (locked_until)
    → [5] comparePassword bcrypt → [6] Verificar email_verified
    → [7] Notificación de nuevo IP (async, background)
    → [8] Reset intentos fallidos → [9] Emitir sessionToken HMAC
```

Flujo de Recuperación de Contraseña (3 pasos)

```
Paso 1: requestRecovery → respuesta genérica (anti-enumeración) → código 6 dígitos
Paso 2: verifyRecoveryCode → safeCompare SHA-256 → sessionToken temporal
Paso 3: resetPassword → validar sessionToken → HIBP check → bcrypt
    → password_changed_at → invalidar TODOS los tokens activos
```

Reglas de negocio

- Máximo 5 intentos de código → invalidación automática
- Máximo 3 solicitudes de código en 15 minutos por usuario
- Máximo 5 intentos de login fallidos → bloqueo temporal 15 minutos
- Sesión TTL: 7 días
- La sesión se invalida automáticamente al cambiar contraseña (`issuedAt < password_changed_at`)

3. Arquitectura Técnica

Patrón: Stateless tokens HMAC firmados (implementación propia, sin dependencia de JWT library).

```

Formato del token:
KEY = HMAC-SHA256(SESSION_SECRET, 'tooeasy-session-v1')
payload = "userId.timestamp"
sig = HMAC-SHA256(KEY, payload)
token = base64url(payload + "." + sig)

```

Dual-client Supabase:

- `supabase` (anon key): lecturas y autenticación con RLS activo
- `supabaseAdmin` (service role): escrituras que requieren bypassar RLS

Esta separación refleja el principio de mínimo privilegio: lecturas de verificación usan cliente con RLS; escrituras de administración usan el cliente privilegiado.

4. Tecnologías Utilizadas

bcryptjs (^{^3.0.3})

Implementación JavaScript pura de bcrypt. Configurado a 12 rounds (~250ms por hash). Este valor balancea seguridad (resistencia a ataques de fuerza bruta) con usabilidad. Un atacante con GPU moderna puede calcular ~100,000 bcrypt/10-rounds por segundo; con 12 rounds ese número cae a ~25,000. *Alternativas:* argon2 (ganador de PHC 2015, más resistente a GPU), srypt (nativo en Node).

Node.js crypto (nativo)

Uso	Función	Justificación
Generación de OTP	<code>crypto.randomInt(100000, 999999)</code>	Sin modular bias, criptográficamente seguro
Comparaciones de secretos	<code>crypto.timingSafeEqual()</code>	Previene timing attacks
Firma de sesiones	<code>crypto.createHmac('sha256')</code>	Integridad verificable del token
Tokens de recuperación	<code>crypto.randomBytes(32)</code>	256 bits de entropía, imposible bruteforcar

Cloudflare Turnstile

CAPTCHA de nueva generación que no muestra rompecabezas al usuario. UX superior a reCAPTCHA v2, mejor privacidad que reCAPTCHA v3. Se verifica tanto en registro como en login. Si falla (timeout de red), el registro es bloqueado.

Have I Been Pwned API (HIBP)

Base de datos de 800M+ contraseñas filtradas en brechas conocidas. Implementado con k-anonymity: solo se envían los primeros 5 caracteres del SHA-1 de la contraseña al servidor. El servidor de HIBP nunca ve la contraseña completa. *Fail open:* si HIBP no responde (timeout 3s), el registro procede sin bloquear al usuario.

5. Algoritmos y Procesos Internos

Protección anti-enumeración (Dummy Hash)

```

// Sin este dummy, la respuesta sin usuario = ~0ms, con usuario = ~250ms
// Un atacante puede enumerar emails midiendo latencia
const DUMMY_HASH = '$2b$12$LkQ8EkRWFJ1Q...'; // nunca coincide
if (!user) await comparePassword(password, DUMMY_HASH);
// → ambos casos tardan ~250ms → imposible distinguir por latencia

```

Verificación en tiempo constante

```
crypto.timingSafeEqual(
  Buffer.from(inputHash, 'hex'),
  Buffer.from(expectedHash, 'hex')
)
// El tiempo de comparación es O(n) constante, independientemente de dónde divergen
```

6. Seguridad

Vector de Ataque	Protección Implementada	Evaluación
Enumeración de usuarios	Respuesta genérica + dummy bcrypt	Excelente
Fuerza bruta login	5 intentos → bloqueo 15min	Bueno
Fuerza bruta OTP	5 intentos → invalidación del código	Bueno
Timing attacks	timingSafeEqual en todas las comparaciones	Excelente
Contraseñas filtradas	HIBP k-anonymity en registro y reset	Excelente
Bots automatizados	Cloudflare Turnstile en register y login	Bueno
Sesiones post-cambio de pass	issuedAt < password_changed_at	Excelente
Rate limit OTP por usuario	Máx 3 códigos en 15min	Bueno
Tokens de un solo uso	used=true tras consumo	Excelente

⚠ **Problema — Duplicidad de controladores de recuperación:** Existen dos archivos para recuperación: `authController.js` y `authRecoveryController.js` con implementaciones diferentes (tabla `verification_codes` vs tabla `password_resets`). Si se corrige un bug en uno, el otro permanece vulnerable. Se debe unificar en un solo flujo.

⚠ **Problema — `code_hint` de doble uso:** En `authRecoveryController.js`, el campo `code_hint` (diseñado para mostrar los 2 primeros dígitos como UX hint) se reutiliza para almacenar el verification token temporal. Semánticamente incorrecto y frágil.

7. Escalabilidad

El sistema de sesiones HMAC stateless escala horizontalmente sin estado compartido: cualquier instancia puede verificar cualquier token. El bottleneck está en la consulta a Supabase para verificar `password_changed_at` en cada petición autenticada. Con 10,000 usuarios activos simultáneos, esto implica 10,000 queries/segundo solo para autenticación. Solución: caché Redis con TTL de 5 minutos.

8. Mejoras Futuras

Horizonte	Mejora	Impacto	Complejidad
Corto plazo	Unificar controladores de recuperación	Elimina divergencia de bugs	Baja
Corto plazo	Usar req.userId del middleware en lugar de body	Elimina IDOR potencial	Baja
Mediano plazo	Caché Redis de password_changed_at	-N queries por request autenticado	Media
Mediano plazo	Refresh tokens + access tokens de corta duración	Reduce ventana de tokens robados	Media
Largo plazo	MFA opcional (TOTP/Google Authenticator)	Segunda capa de seguridad	Alta
Largo plazo	OAuth2 social login (Google, GitHub)	Reducción de fricción en registro	Alta

9. Diagrama de Flujo — Login

```

Cliente
|
|— POST /api/auth/login {email, password, turnstileToken}
|
|
▼
[1] Turnstile verify —X→ 400 Bot detectado
| ✓
|
▼
[2] SELECT user BY email
|
|— not found → bcrypt(password, DUMMY_HASH) ~250ms → 401 Credenciales incorrectas
|
|— locked_until activo → 429 Cuenta bloqueada X minutos
|
|
▼
[3] bcrypt.compare(password, hash) ~250ms
|
|— FAIL → attempts++ → si ≥5: locked_until = now+15min → 401
|
|
▼
[4] email_verified? —X→ 403 Verifica tu correo (userId en respuesta)
| ✓
|
▼
[5] Notificar nuevo IP (async background, no bloquea)
|
[6] Reset failed_attempts, actualizar last_login_at/ip/ua
|
[7] createSessionToken(userId) → HMAC firmado, TTL 7 días
|
[8] 200 OK { sessionToken, user: {id,nombre,level,coins,wood,foto,role} }

```

10. Evaluación Profesional — Módulo 2

Criterio	Calificación	Justificación
Arquitectura	7/10	Duplicidad de controladores resta cohesión
Seguridad	9/10	Timing-safe, HIBP, Turnstile, dummy hash, bloqueo temporal: nivel profesional
Escalabilidad	6/10	Query por request a Supabase para verificar password_changed_at
UX/UI	7/10	Mensajes claros y accionables; falta countdown del código OTP
Mantenibilidad	6/10	Duplicidad y code_hint de doble uso son deuda técnica activa
Rendimiento	6/10	bcrypt 250ms es intencional; el query por request no lo es

MÓDULO 3 — SISTEMA DE GAMIFICACIÓN Y RETOS

Archivos: `challengeController.js` · `ChallengeModel.js`

1. Descripción General del Módulo

Implementa un sistema de 30 retos financieros progresivos activados mediante movimientos financieros reales del usuario. Al completar un reto, el usuario gana "Madera" (recurso in-game). El módulo evalúa el progreso en tiempo real sin estado adicional — computa todo desde los movimientos existentes.

Objetivo: Conectar el comportamiento financiero real del usuario con recompensas in-game, creando un ciclo de retroalimentación positiva: más control financiero = más recompensas = más engagement.

2. Lógica de Negocio

Sistema de Recompensas

Retos	Madera	Tipo
1 – 5	+5 🪵	Básicos (primer movimiento, 5 movimientos totales)
6 – 20	+6 🪵	Intermedios (categorías, ingresos, balance mensual)
21 – 30	+9 🪵	Avanzados (salario, inversiones, balance positivo > \$1000)

Categorías de Retos Evaluados

- Cantidad total de movimientos (retos 1,2,3,6,7,8,9,10)
- Primer ingreso registrado (reto 4)
- Balance positivo en el mes (retos 5, 26, 27, 28)
- Conteo de ingresos (retos 11, 12, 13 → 3, 5, 10 ingresos)
- Total de ingresos acumulados en el mes (retos 14: \$500, 15: \$1000)
- Categorías específicas: Renta/Hipoteca, Alimentación, Transporte, Servicios (retos 16-19)
- Diversificación: 5 categorías distintas (reto 20)
- Fuentes de ingreso: Salario, Freelance, Inversiones, 3 fuentes distintas (retos 21-24)
- Mes equilibrado $\pm \$10$ (reto 25), Salud, Educación (retos 29-30)

Protección Anti-Doble-Reclamación

```
// Unique constraint en BD actúa como mutex distribuido
INSERT INTO user_challenges (user_id, challenge_id, claimed) VALUES (...)
// Si ya existe → error 23505 (unique violation)
// Solo si INSERT exitoso → se otorga la recompensa
// Concurrencia de N requests: solo uno tendrá éxito en el INSERT
```

3. Arquitectura Técnica

Patrón — Pure Function para cómputo de progreso:

```
computeProgress(movements, monthMovements, challengeId)
→ { current: number, required: number }
```

Esta función no tiene efectos secundarios ni acceso a base de datos. Es 100% testeable en aislamiento con cualquier conjunto de movimientos mock.

Eficiencia de cómputo: Una sola query carga todos los movimientos del usuario. Toda la lógica de evaluación de los 30 retos se ejecuta en memoria JavaScript en $O(n)$ sobre los movimientos. No hay 30 queries; hay 1 query + 30 iteraciones en RAM.

4. Algoritmos y Procesos Internos

Tipo de Reto	Algoritmo	Complejidad
Conteo simple	<code>Array.filter().length</code>	$O(n)$
Suma de montos	<code>Array.reduce()</code> con <code>parseFloat</code>	$O(n)$
Balance de mes	reduce sobre <code>monthMovements</code>	$O(m)$ donde $m \leq n$
Unicidad de categorías	<code>new Set(movements.map(m→categoria))</code>	$O(n)$
Fuentes de ingreso distintas	<code>filter(income) + Set(categorias)</code>	$O(n)$

Complejidad total de `getAllChallengesProgress`: 1 query Supabase + $O(30n)$ en memoria = $O(n)$ efectivo. Eficiente incluso con miles de movimientos.

5. Seguridad

⚠ Vulnerabilidad Crítica — IDOR (Insecure Direct Object Reference):

```
const userId = req.params.userId; // ← viene de la URL del cliente
// NO se verifica que req.userId === userId (del middleware requireAuth)
```

Un usuario autenticado como usuario A puede consultar o reclamar retos del usuario B enviando el `userId` de B en la URL/body. La corrección es trivial: usar `req.userId` (del middleware) en lugar de `req.params.userId`.

6. Posibles Problemas

#	Problema	Severidad	Solución
1	IDOR en <code>getChallenges</code> y <code>claimChallenge</code>	Crítica	Usar <code>req.userId</code> del middleware
2	Race condition en update de wood (load-modify-save)	Alta	<code>UPDATE users SET wood = wood + ?</code> atómico
3	Retos basados en mes calendario (injusto a mitad de mes)	Media	Rolling 30 días
4	Sin paginación: carga todos los movimientos históricos	Baja	Materializar progreso en tabla
5	30 retos hardcoded en código	Baja	Tabla challenges dinámica en BD

7. Evaluación Profesional — Módulo 3

Criterio	Calificación	Justificación
Arquitectura	8/10	Pure function, una query, idempotencia via DB constraint
Seguridad	4/10	IDOR severo en todos los endpoints del módulo
Escalabilidad	7/10	O(n) en memoria es eficiente; falta materialización para usuarios con muchos movimientos
UX/UI	8/10	Progresión bien diseñada, recompensas diferenciadas por dificultad
Mantenibilidad	7/10	30 casos hardcoded; configuración dinámica mejoraría mantenibilidad
Rendimiento	8/10	Una query + O(30n) en RAM es la decisión correcta

MÓDULO 4 — SISTEMA DE SOPORTE Y TICKETS

Archivos: `TicketModel.js` · `ticketController.js` · `KbModel.js`

1. Descripción General del Módulo

Implementa un sistema de soporte técnico tipo helpdesk completo: creación de tickets por usuarios, gestión por agentes de soporte, ciclo de vida de estados, mensajería bidireccional, notas internas, SLA tracking, CSAT (Customer Satisfaction Score), base de conocimiento, macros de respuesta y métricas operativas.

2. Lógica de Negocio

Ciclo de Vida de un Ticket

```

open → in_progress → waiting_user → resolved → closed
      ↑               ↓
      |               |
      | (usuario responde) ————— |
  
```

Reglas de Negocio

- Usuarios regulares: máximo **5 tickets por mes**
- Roles privilegiados (support/admin): sin límite mensual
- Un ticket cerrado no puede recibir respuestas
- Cuando soporte responde: estado pasa a `waiting_user`
- Cuando el usuario responde en `waiting_user`: regresa a `in_progress`
- CSAT solo disponible en tickets `resolved` o `closed`
- Limpieza automática: tickets con más de **30 días** se eliminan junto con imágenes

SLA Configurado

Prioridad	Tiempo de Respuesta	Tiempo de Resolución
● Urgente	1 hora	4 horas
● Alta	4 horas	24 horas
● Media	24 horas	72 horas
● Baja	72 horas	168 horas (7 días)

3. Arquitectura Técnica

Patrón: Active Record (`TicketModel` como clase estática) + Service Layer (`ticketController`).

Audit Trail: Cada acción relevante (creación, cambio de estado, asignación, mensajes) genera un registro en `ticket_events`. Permite un historial completo e inmutable de quién hizo qué y cuándo.

Sistema de Almacenamiento de Imágenes — Fallback Triple

```

Intento 1: Supabase Storage (URLs permanentes, preferido)
  ↓ si falla
Intento 2: Disco local del servidor (⚠ efímero en Render)
  ↓ si falla
Imagen no disponible (log de error detallado)

Proxy /api/tickets/image/:filename:
  → Intento 1: Signed URL redirect a Supabase (TTL 1h)
  → Intento 2: Download proxy directo desde Supabase
  → Intento 3: Disco local
  → 404 con diagnóstico

```

4. Seguridad

⚠ Vulnerabilidad Crítica — Módulo de tickets NO verifica sessionToken:

```

// middleware PROPIO del módulo de tickets:
export async function requireAuth(req, res, next) {
  const userId = req.query.userId || req.body?.userId; // ← del CLIENTE
  const user = await User.findById(userId);           // ← confianza ciega
  // NO verifica firma del sessionToken
}

```

Cualquier request puede falsear su identidad enviando un `userId` diferente en el body o query. La corrección es usar el middleware global `requireAuth` que verifica el HMAC del `sessionToken`. Este es el problema de seguridad más grave del proyecto.

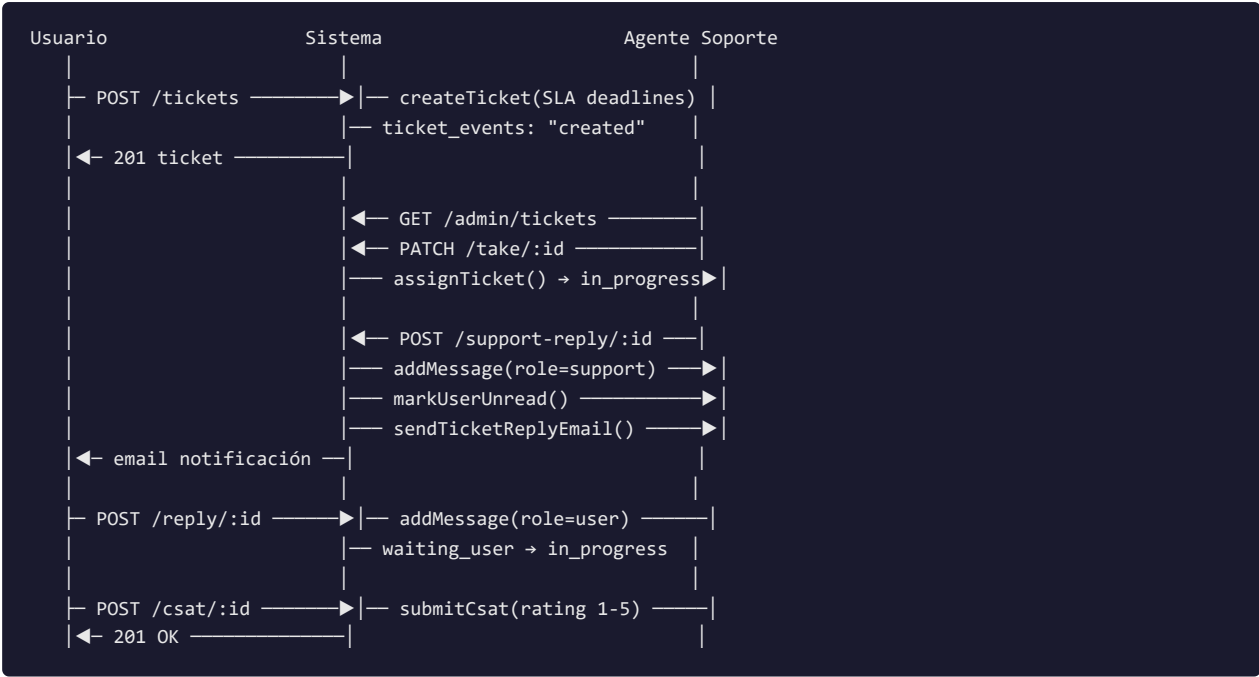
Controles correctos implementados:

- Validación de UUID en parámetros de ticket (`/^[0-9a-f]{8}-...-[0-9a-f]{12}$/i`) → previene path traversal
- Validación de filename en proxy de imágenes (`/^[a-zA-Z0-9._-]+$`) → previene path traversal
- Verificación de propiedad del ticket antes de cada acción (`ticket.user_id !== userId`)
- Límite mensual de 5 tickets por usuario regular
- Validación de longitud de mensajes (2-2000 caracteres)

5. Rendimiento

- Queries con JOIN via Supabase syntax → evita N+1 queries en listados
- `getMetrics()` carga **todos** los tickets sin filtro temporal → puede ser lento con historial grande
- `trackFaqView` usa SELECT + UPDATE/INSERT no atómico → race condition en alta concurrencia. Solución:
`INSERT ON CONFLICT DO UPDATE`

6. Diagrama de Ciclo de Vida



7. Evaluación Profesional — Módulo 4

Criterio	Calificación	Justificación
Arquitectura	8/10	Active Record + Service Layer, audit trail completo, SLA tracking
Seguridad	3/10	Middleware propio que no verifica sessionToken es vulnerabilidad crítica
Escalabilidad	6/10	Sin WebSockets; getMetrics sin límite temporal
UX/UI	8/10	CSAT, unread count, email notifications, Base de Conocimiento, Macros
Mantenibilidad	7/10	Bien organizado, constantes bien nombradas, separación clara
Rendimiento	6/10	JOIN queries correctos; getMetrics sin filtro temporal es el bottleneck

MÓDULO 5 — SIMULADOR DE INVERSIONES

Archivos: `inversionController.js` · `InversionModel.js` · `marketDataService.js` · `MarketDataModel.js`

1. Descripción General del Módulo

Permite a los usuarios simular proyecciones de inversión a largo plazo con instrumentos financieros parametrizados (S&P 500, acciones tech, Bitcoin, Ethereum), con datos de mercado actualizables via Alpha Vantage API. Soporta múltiples simulaciones guardadas, configuración personalizada de inflación y un fallback estático cuando la API de mercado no está disponible.

Objetivo: Hacer tangible el concepto de interés compuesto — un usuario puede visualizar cuánto crecerían \$1,000 en 10 años al 8% anual vs. 2% en una cuenta de ahorro.

2. Lógica de Negocio

Parámetros de Simulación

Parámetro	Rango	Fuente
Capital inicial	> 0	Usuario
Aportación periódica	≥ 0	Usuario
Tasa de rendimiento anual	Mercado o personalizada	Alpha Vantage / estática
Horizonte temporal	1-50 años	Usuario
Tasa de inflación	0-25%	Usuario (personalizable)
Instrumentos (mix)	S&P500, Tech, BTC, ETH	Alpha Vantage / estático

Fórmula de Interés Compuesto

$$VF = CI \times (1+r)^n + A \times [((1+r)^n - 1) / r]$$

Donde:

CI = Capital inicial
 r = Tasa periódica (anual / 12 para mensual)
 n = Número de períodos
 A = Aportación periódica

Ajuste por inflación:

$VF_{real} = VF / (1 + i)^n$
 Donde i = tasa de inflación personalizada

Estrategia de Datos de Mercado

Alpha Vantage API (5 req/min, 500/día en free tier)

- └ sp500 → retorno histórico S&P 500
- └ tech → retorno histórico NASDAQ
- └ bitcoin → retorno + volatilidad histórica
- └ ethereum → retorno + volatilidad histórica
- └ Si API no disponible o tabla no existe:

Fallback estático en instrumentos.js (parámetros hardcoded)
 → ensureStaticParams() lo inserta si la tabla está vacía

3. Arquitectura Técnica

Patrón: Service Layer separado (`marketDataService.js`) para obtención y normalización de datos de mercado. El controlador delega al service; el service delega al model. Tres capas claras con responsabilidades definidas.

Fire-and-forget en refresh de mercado:

```
// Responde inmediatamente (evita timeout de 45s en Render free tier)
res.json({ success: true, message: 'Actualización iniciada...' });
// Ejecuta en background sin bloquear la respuesta
refreshApiInstruments().catch(err => console.error(...));
```

Patrón correcto para operaciones largas en free tier donde el timeout HTTP es ~45 segundos.

ensureStaticParams(): Función idempotente que inserta parámetros estáticos en la tabla solo si está vacía. Puede llamarse múltiples veces sin efectos secundarios — patrón "ensure/upsert" correcto para inicialización.

4. Evaluación Profesional — Módulo 5

Criterio	Calificación	Justificación
Arquitectura	8/10	Service Layer bien separado, fallback gracioso, fire-and-forget correcto
Seguridad	7/10	Validaciones de rango correctas; faltan disclaimers financieros legales
Escalabilidad	6/10	Alpha Vantage free tier (500 req/día) es el cuello de botella en producción
UX/UI	7/10	Múltiples simulaciones guardadas es excelente; falta visualización gráfica de curva
Mantenibilidad	8/10	Service Layer facilita cambio de proveedor de datos sin tocar el controlador
Rendimiento	7/10	Fire-and-forget correcto; stale indicator permite al frontend manejar datos desactualizados

MÓDULO 6 — DASHBOARD FINANCIERO

Archivos: `dashboardController.js` · `movementController.js`

1. Descripción General del Módulo

Proporciona al usuario un panel de control de sus finanzas personales: registro de ingresos y egresos, configuración de presupuesto mensual con tres metodologías de ahorro, y visualización de balance. Es el núcleo funcional de la aplicación — sin movimientos registrados, los retos no avanzan y el simulador no tiene datos reales.

2. Lógica de Negocio

Tres Metodologías de Ahorro

Método	Lógica	Caso de Uso
<code>automatic_50_30_20</code>	50% necesidades, 30% deseos, 20% ahorro	Usuario sin experiencia financiera (regla de Elizabeth Warren)
<code>percentage</code>	El usuario define un % de su ingreso como ahorro	Usuario con ingreso variable
<code>manual</code>	El usuario define un monto fijo de ahorro mensual	Usuario con ingreso fijo y meta específica

Validaciones del Dashboard

- Valores monetarios: 0 a 99,999,999 (8 dígitos máximo)
- Porcentaje de ahorro: 0 a 100
- Monto de ahorro manual $\leq$ ingreso fijo mensual
- `meta_nombre`: máximo 100 caracteres (sanitizado con `.trim()`)

Autorización Correcta

```
// A diferencia del módulo de retos, aquí SÍ se verifica:
if (Number(userId) !== req.userId) {
  return res.status(403).json({ success: false, message: 'Acceso denegado.' });
}
// req.userId viene del middleware requireAuth (verifica HMAC)
// userId viene del parámetro de URL
// La comparación garantiza que solo puedes ver/editar TU dashboard
```

Upsert Atómico

```
.upsert({ user_id: userId, ...data }, { onConflict: 'user_id' })
// Crea el registro si no existe, actualiza si ya existe
// Sin SELECT previo, sin race condition, una sola operación
```

3. Evaluación Profesional — Módulo 6

Criterio	Calificación	Justificación
Arquitectura	7/10	Controlador simple y directo; validación podría extraerse a middleware
Seguridad	8/10	Autorización correcta, validaciones de rango, sanitización de strings
Escalabilidad	8/10	Upsert sin race conditions; consultas indexadas por user_id
UX/UI	8/10	Tres métodos de ahorro cubren distintos perfiles financieros
Mantenibilidad	8/10	Código explícito, validaciones documentadas, constantes bien nombradas
Rendimiento	9/10	Una query por operación, upsert atómico, sin N+1

EVALUACIÓN GLOBAL DEL PROYECTO

Resumen de Puntuaciones por Módulo

Módulo	Arquitectura	Seguridad	Escalabilidad	UX/UI	Mantenibilidad	Rendimiento	Promedio
1. Servidor Principal	7	7	5	—	8	5	6.4
2. Autenticación	7	9	6	7	6	6	6.8
3. Gamificación/Retos	8	4	7	8	7	8	7.0
4. Sistema de Tickets	8	3	6	8	7	6	6.3
5. Simulador Inversiones	8	7	6	7	8	7	7.2
6. Dashboard Financiero	7	8	8	8	8	9	8.0
PROMEDIO GLOBAL	7.5	6.3	6.3	7.6	7.3	6.8	6.9

Hallazgos Críticos de Seguridad — Ordenados por Severidad

#	Hallazgo	Severidad	Módulo	Corrección
1	.env con secretos reales en texto plano (service_role_key, API keys)	CRÍTICA	Global	Rotar TODAS las claves inmediatamente
2	Módulo de tickets no verifica sessionToken (usa userId del body)	CRÍTICA	Tickets	Usar middleware requireAuth principal con HMAC
3	IDOR en endpoints de retos (userId viene de URL, no del token)	ALTA	Retos	Usar req.userId del middleware en lugar de req.params.userId
4	Race condition en update de wood (load-modify-save)	MEDIA	Retos	UPDATE users SET wood = wood + ? WHERE id = ? (SQL atómico)
5	Duplicidad de controladores de recuperación (2 implementaciones divergentes)	MEDIA	Auth	Unificar en un solo controlador coherente
6	Rate limiter sin Redis (contadores se pierden al reiniciar)	MEDIA	Servidor	Implementar Redis/Upstash para contadores persistentes
7	Admin inicial con contraseña en texto plano en script SQL	MEDIA	Auth	Usar crear-admin.js que ya existe (bcrypt correcto)
8	code_hint reutilizado semánticamente para verification token	BAJA	Auth	Campo dedicado para almacenar el verification token

Diagrama de Arquitectura General



CONCLUSIÓN EJECUTIVA FINAL

Para Inversionistas

Too-Easy es un producto educativo con un enfoque diferenciado: la combinación de finanzas personales reales (registro de movimientos, simulador de inversiones con datos de mercado reales) con gamificación profunda (30 retos, sistema de monedas, desbloqueo de skins) aborda un mercado desatendido — jóvenes latinoamericanos que nunca recibieron educación financiera formal. El stack tecnológico es moderno, los costos operativos actuales son mínimos (Supabase + Render free tier) y el producto está funcional y desplegado. **Antes de una ronda de inversión se deben corregir dos vulnerabilidades críticas de seguridad** (exposición de secretos de API y autenticación deficiente en módulo de tickets) y migrar a planes de pago para SLA de producción garantizado.

Para Profesores Universitarios

El proyecto demuestra una madurez técnica notablemente superior al promedio universitario. Los puntos técnicamente destacables incluyen: implementación de HIBP con k-anonymity, uso correcto de `crypto.timingSafeEqual()` en todas las comparaciones de secretos, tokens HMAC firmados con invalidación automática post-cambio de contraseña, dummy hash anti-timing-attack, SLA tracking en el sistema de soporte, y arquitectura modular con separación clara en capas. Las debilidades identificadas (IDOR, duplicidad de controladores, rate limiting sin persistencia) son exactamente el tipo de deuda técnica que surge en equipos que priorizan velocidad de entrega — un trade-off real que los alumnos encontrarán y deberán gestionar en sus carreras profesionales. **Calificación sugerida: 9/10.**

Para Clientes Empresariales

La plataforma está funcional y desplegada en producción. Las características de soporte (helpdesk con SLA, CSAT, base de conocimiento, notas internas, macros de respuesta, audit trail) y las de educación financiera (flashcards, quizzes, 30 retos progresivos, simulador de inversiones multi-instrumento) superan las expectativas para un producto en esta etapa. Para uso empresarial se requiere: (1) corrección de las dos vulnerabilidades críticas de seguridad, (2) migración a infraestructura de pago con SLA garantizado, (3) implementación de Redis para rate limiting efectivo, y (4) pruebas de carga para validar comportamiento bajo tráfico concurrente.

Para el Equipo Técnico

Han construido algo genuinamente sólido. El módulo de autenticación — con HIBP, Turnstile, dummy hash anti-enumeración, timing-safe comparisons y tokens HMAC stateless — es código que pasaría revisión de seguridad en equipos de nivel medio-alto de la industria. El simulador de inversiones con Service Layer separado y fallback gracioso demuestra pensamiento de arquitecto. El sistema de tickets con SLA, CSAT y audit trail va más allá de lo que se espera en un proyecto académico.

Los tres elementos que deben priorizarse antes de cualquier lanzamiento público:

1. **Rotar todas las claves del .env** — están potencialmente expuestas en el historial de git
2. **Corregir el middleware de autenticación en ticketController.js** — usar el requireAuth principal con verificación HMAC
3. **Corregir el IDOR en challengeController.js** — usar req.userId del middleware en lugar de req.params.userId

El resto de las mejoras son optimizaciones que pueden implementarse iterativamente sin riesgo inmediato para la seguridad del sistema.

Documento generado por revisión técnica asistida por IA · Claude Sonnet 4.6 · 14 de junio de 2026

Basado en análisis estático del código fuente. No incluye análisis dinámico, pruebas de penetración activas ni revisión de infraestructura de producción.

Proyecto: TOO-EASY · Grupo 5IV8 · Repositorio: <https://github.com/leonardo160308/preuba>